Helper functions

Aditya Sarkar

Tulane Dr, Hyattsville, MD | November 11, 2026

1 `_positive_int(v, what)`

1.1 Inputs and Preconditions

- `v` (`Any`): The target value to validate.
- **`what`** (`str`): The identifier or parameter label used for formatting error messages.

1.2 Internal Logic and Validation

The function strictly guards against non-integer types, negative values, zero, and Python booleans (since `isinstance(True, int)` is truthy in Python):

$$\text{Valid}(v) \iff (v \notin \{\text{True}, \text{False}\}) \land (v \in \mathbb{Z}) \land (v > 0) \tag{1}$$

If any predicate fails, it raises:

```
CaptureError(f"{what} is {v!r}, which is not a positive integer.")
```
(2)

1.3 Outputs

- **Return Value** (`int`): The validated strictly positive integer v.

2 `dbfs(amplitude)`

2.1 Inputs and Preconditions

- **`amplitude`** (`float`): Normalized magnitude a crossing the module boundary, where $a \in [0.0, 1.0]$. Negative inputs are rejected as raw signed samples that do not express magnitude.

2.2 Mathematical Formulation

Decibels relative to Full Scale (dBFS) quantify signal level relative to the maximum permissible digital saturation limit ($1.0 \equiv 0$ dBFS):

$$\text{dbfs}(a) = \begin{cases} 20 \log_{10}(a), & \text{if } a > 0 \\ \text{None}, & \text{if } a = 0.0 \end{cases} \tag{3}$$

Handling Digital Silence: The analytical limit of decibels approaching digital zero is:

$$\lim_{a \to 0^+} 20 \log_{10}(a) = -\infty \tag{4}$$

Returning `None` avoids non-serializable IEEE floating-point values (`-inf`) in downstream formats like JSON, forcing caller routines to handle absolute silence explicitly.

2.3 Outputs

- **Return Value** (`float | None`): Level in dBFS or `None` if $a = 0.0$.

3 `level_report(peak, rms, clipped, samples)`

3.1 Inputs and Preconditions

Table 1: Input Parameters for `level_report`

Parameter	Symbol	Type	Description / Constraints
`peak`	p	`float`	Normalized peak magnitude $\in [0.0, 1.0]$.
`rms`	r	`float`	Normalized RMS magnitude $\in [0.0, 1.0]$.
`clipped`	c	`int`	Non-negative count of clipped samples $(0 \leq c \leq N)$.
`samples`	N	`int`	Strictly positive sample count $(N \in \mathbb{Z}_{>0})$.

3.2 Internal Processing and Assertions

3.2.1 Invariant Check: L_2 vs. L_∞ Norms

For any discrete digital signal vector $\mathbf{x} = [x_1, \ldots, x_N]^T \in \mathbb{R}^N$:

$$p = \|\mathbf{x}\|_\infty = \max_{1 \leq i \leq N} |x_i|, \qquad r = \frac{1}{\sqrt{N}} \|\mathbf{x}\|_2 = \sqrt{\frac{1}{N} \sum_{i=1}^{N} x_i^2} \tag{5}$$

By Cauchy-Schwarz and standard norm bounds, $r \leq p$. If $r > p$, the measurement arrays are inconsistent or corrupted:

$$r > p \implies \text{Raise } \texttt{CaptureError} \tag{6}$$

3.2.2 Crest Factor Calculation

Measures dynamic range from continuous power to instantaneous maximum:

$$\text{crest} = \begin{cases} p_{\text{db}} - r_{\text{db}} = 20 \log_{10}\left(\frac{p}{r}\right), & \text{if } p_{\text{db}} \neq \text{None} \wedge r_{\text{db}} \neq \text{None} \\ \text{None}, & \text{otherwise} \end{cases} \tag{7}$$

3.2.3 Clipping Rate

$$\text{rate} = \frac{c}{N}, \quad \text{rate} \in [0.0, 1.0] \tag{8}$$

3.2.4 Threshold and Usability Logic

$$\text{silent} \iff p = 0.0 \tag{9}$$

$$\text{too_hot} \iff (p_{\text{db}} \neq \text{None} \wedge p_{\text{db}} > \text{PEAK_DBFS_MAX}) \vee (\text{rate} > \text{CLIP_RATE_MAX}) \tag{10}$$

$$\text{too_quiet} \iff \text{silent} \vee (p_{\text{db}} \neq \text{None} \wedge p_{\text{db}} < \text{PEAK_DBFS_MIN}) \tag{11}$$

$$\text{usable} \iff \neg(\text{silent} \vee \text{too_hot} \vee \text{too_quiet}) \tag{12}$$

3.3 Outputs

Returns a dictionary containing:

Table 2: Dictionary Schema Returned by `level_report`

Key	Type	Meaning
`"peak"`	`float`	Evaluated peak magnitude $p \in [0.0, 1.0]$.
`"rms"`	`float`	Evaluated RMS magnitude $r \in [0.0, 1.0]$.
`"peak_dbfs"`	`float | None`	Full-scale level p_{db}.
`"rms_dbfs"`	`float | None`	Full-scale level r_{db}.
`"crest_db"`	`float | None`	Crest factor $p_{\text{db}} - r_{\text{db}}$.
`"clipped"`	`int`	Number of rail-saturated samples c.
`"samples"`	`int`	Total samples inspected N.
`"clip_rate"`	`float`	Fraction of clipped samples $\frac{c}{N}$.
`"silent"`	`bool`	True if $p = 0.0$.
`"too_hot"`	`bool`	True if signal clipped excessively or breached maximum peak.
`"too_quiet"`	`bool`	True if signal is silent or below minimum peak threshold.
`"usable"`	`bool`	Conjunction: $(\neg\text{silent}) \wedge (\neg\text{too_hot}) \wedge (\neg\text{too_quiet})$.

4 `level_sentence(level)`

4.1 Inputs and Preconditions

- `level` (`dict[str, Any]`): The evaluation dictionary emitted by `level_report`.

4.2 Internal Logic

1. If `level["silent"]` is true, directly format:

   "the capture is digitally silent: all N samples are zero"

2. Otherwise, build a comma-separated list of formatted metric strings:
   - Peak: `"peak {`p_{db}`:.2f} dBFS"`
   - RMS (if present): `"RMS {`r_{db}`:.2f} dBFS"`
   - Crest (if present): `"crest {crest:.2f} dB"`
   - Clipping (if $c > 0$): "c `samples at the rail (100.0 ×` `rate:.4f%)"`

4.3 Outputs

- **Return Value** (`str`): A single-line human-readable summary of the level report.

5 `bits_in_use(peak_dbfs, bit_depth)`

5.1 Inputs and Preconditions

- **`peak_dbfs`** (`float | None`): Peak level from `dbfs(p)`.
- **`bit_depth`** (`int`): Bit resolution per sample (e.g., 16 or 24), validated via `_positive_int`.

5.2 Mathematical Formulation

Calculates the effective number of quantizer bits spanned by the signal's maximum excursion:

$$B_{\text{used}} = \begin{cases} 0.0, & \text{if } p_{\text{db}} \text{ is None} \\ \max\left(0.0, B_{\text{depth}} + \frac{p_{\text{db}}}{\text{DB_PER_BIT}}\right), & \text{if } p_{\text{db}} \in \mathbb{R} \end{cases} \tag{13}$$

where $\text{DB_PER_BIT} = 20\log_{10}(2) \approx 6.0206$ dB/bit.

5.3 Outputs

- **Return Value** (`float`): Number of utilized quantizer bits.